
Search Space Synthesis for Parametric Functions

Felix Laarmann^{1,2} Andreas Pauly^{1,2} Sebastian Buschjäger² Andrea Bommert³ Jakob Rehof^{1,2}

Abstract

We present a general framework for synthesizing search spaces of parametric functions, along with strategies for traversing these spaces to find optima. We formalize an algebraic theory for the categorical model of parametric functions in finite combinatory logic with predicates (FCLP). Based on a component-oriented synthesis framework for FCLP we automate composition from given components and search for parametric functions. Components are language-agnostic and may be instantiated as any implementation of parametric functions, e.g., as PyTorch modules. A proof-of-concept implementation demonstrates how to represent more specific concepts, such as neural architecture search and hyperparameter optimization, within the framework.

1. Introduction

With the proliferation of machine learning (ML) techniques and their success across many application domains, there is a natural interest in understanding the essential structures underlying various classes of machine learning models from an abstract mathematical perspective. In particular, abstract models of classical ML techniques, such as backpropagation, have recently been proposed within category theory (Fong et al., 2019; Cruttwell et al., 2022; Gavranović et al., 2024). Inspired by these developments, we propose to put these abstract models to use by automating the construction of model classes and their implementations using program synthesis techniques (David & Kroening, 2017). We provide a highly general and flexible framework for automated *search space synthesis* for classes of models, focusing on *parametric functions*, along with strategies for traversing them to find optima. A useful characteristic of categorical

models is that they are, by nature, *compositional*. Using a framework based on typed combinatory logic FCLP (finite combinatory logic with predicates (Dudenhefner et al., 2023)) for component-oriented synthesis (Rehof & Vardi, 2014), we can exploit compositional structure to decompose families of complex models into simpler building blocks (components, represented as combinators in combinatory logic), enabling component reuse and simplifying specification. We provide a proof-of-concept implementation that showcases how neural architecture search (NAS) and hyperparameter optimization (HPO) methods arise naturally as implementations within our framework, on the one hand, and how our approach provides the flexibility to synthesize and explore novel (to the best of the authors’ knowledge) search spaces for ML models, on the other hand.

Related Work: Existing approaches, such as BOHNAS (Schrodi et al., 2023) and Einspace (Ericsson et al., 2024), use extensions of context-free grammars (CFGs) as search spaces. These approaches require users to write the grammars for search spaces, whereas our approach provides a specification language and synthesizes search spaces represented as tree languages, along with search strategies for a given specification. In contrast to manually designed architecture grammars, the proposed approach automatically derives search spaces from typed component specifications while guaranteeing the structural well-formedness of synthesized models. Our approach aims to achieve higher degrees of automation and greater ease of use for the ML community. The FCLP framework (Dudenhefner et al., 2023) can represent solution spaces that go beyond context-free structures by (automatically) representing them as Horn-theories, although the present application is confined to tree languages. Closely related in spirit to ours is the work (Negrinho et al., 2019), which defines a domain-specific language for NAS search spaces over computational graphs and their components. The main differences to our approach are that our search spaces are automatically synthesized from component specifications, and specifications are rooted in a more general categorical model. Moreover, our components are typed so that synthesized solutions are guaranteed to be well-formed with respect to the categorical model.

In their ICML 2024 position paper (Gavranović et al., 2024), the authors propose using a structure called the **Para**-construction to represent parametric functions in a

¹Department of Computer Science, TU Dortmund University, Dortmund, Germany ²Lamarr Institute, Dortmund, Germany ³Department of Statistics, TU Dortmund University, Dortmund, Germany. Correspondence to: Felix Laarmann <felix.laarmann@udo.edu>.

generic way. It was proposed that one should search for structures *within* the **Para**-construction to identify categorical datatypes for different ML concepts. This may provide specific categorical models for various ML concepts and enable researchers to reason about them. However, if the primary goal is not to reason about models but to synthesize software that implements them (as in our case), the fact that each of these structures is itself a parametric function (since categories are closed under composition) turns out to be useful, because modern frameworks like PyTorch, TensorFlow, etc., usually provide software components that already implement various ML concepts as parametric functions. A contribution of this paper is to leverage the internal structure of the **Para**-construction *itself* to provide a data structure for (general) parametric functions at the abstraction level of modern ML frameworks. Therefore, it may be viewed as a generalization of the position in (Gavranović et al., 2024) and as a strategy for applying categorical models in modern ML practice. To our knowledge, a specialization of the **Para**-construction to the so-called **Para(Lens)**-construction was first implemented in (Cruttwell et al., 2022) to demonstrate its applicability to learning neural networks.

In their ICML 2010 paper (Liang et al., 2010), the authors employ untyped combinatory logic to program learning in an interesting way, which, however, differs significantly from our use of combinatory logic. They use combinators to represent program transformations on learned programs, whereas we use combinators to represent components and generate families of programs that implement classes of ML pipelines. Moreover, our combinatory theory is typed, which is essential in our context for component- and search space specification and for ensuring the well-formedness of the generated models.

2. A Compositional Theory of Parametric Functions

In this section, we present an algebraic formulation of the **Para**-construction using string diagram constructors that will later serve as the basis for search space synthesis. The **Para**-construction defines a symmetric strict monoidal bicategory of parametric functions (Jia et al., 2025; Gavranović, 2024; Capucci et al., 2022; Gavranović et al., 2024) and provides a compositional theory for parametric functions. We will formalize the **Para**-construction as an algebraic theory by defining constructors for string diagrams, which represent morphisms in the **Para**-construction and, therefore, parametric functions.

To keep the topic accessible, we leave the mathematical details to Appendix A and present an algebraic theory of string diagrams for the **Para**-construction. This algebraic theory formalizes parametric functions as terms of string diagram constructors and term equalities, derived from the categorical definition of the **Para**-construction. The for-

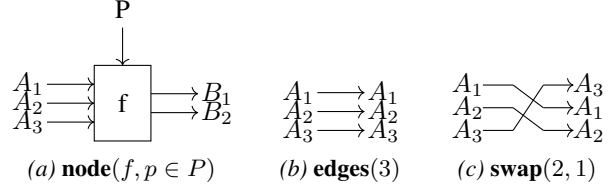


Figure 1. Node, edges, and swap constructors

malization follows the initial algebra approach of Gibbons (Gibbons, 1995) to construct an initial object in the category of enriched symmetric strict monoidal categories.

Following the categorical definitions, we explicitly distinguish between a product $P \otimes Q$ on parameters, a product $A \odot B$ on the domains and codomains of parametric functions, and a parameterization operation $\bullet$. They can, but do not necessarily need to be of the same product construction. A parametric function $f : A \bullet P \rightarrow B$ is a function from A to B parameterized over P . Note that every function can be viewed as a parametric function by choosing the parameter space to be the neutral element of $\bullet$ (usually the singleton set).

Consider linear transformations as an example. They are parametric functions $a : \mathbb{R}^n \bullet \mathbb{R}^{n \times m} \rightarrow \mathbb{R}^m$, implemented as matrix multiplication.

A parametric function $f : (\odot_{i \in [1, n]} A_i) \bullet P \rightarrow (\odot_{i \in [1, m]} B_i)$ corresponds to a node in a string diagram labeled with its identifier f (see Figure 1a), with its domain as n ingoing edges from the left, its codomain as m outgoing edges to the right, and its parameters as an ingoing edge from above. If a function is parameterized by the neutral element of $\bullet$, we may omit the parameter edge. Note that n and m may be 0, corresponding to a node with no ingoing or outgoing edges. The string diagram constructor $\text{node}(f, p) : D_{(n, m)}$ is indexed by the function identifier f and its choice of parameters $p \in P$. $D_{(n, m)}$ is the type of string diagrams with n ingoing and m outgoing edges.

Now that we have defined the nodes of string diagrams, we define their edges. The string diagram constructor for n parallel edges is $\text{edges}(n) : D_{(n, n)}$. Identity functions $\text{id}(\odot_{i \in [1, n]} A_i) : (\odot_{i \in [1, n]} A_i) \rightarrow (\odot_{i \in [1, n]} A_i)$ correspond to the edges of a string diagram; see Figure 1b. Note that identity functions do not allow n to be zero, because $\text{edges}(0)$ encodes the empty string diagram, represented by the constructor $\epsilon : D_{(0, 0)}$.

We are allowed to cross edges. However, we must do so explicitly by constructing a string diagram with $\text{swap}(n, m) : D_{(n+m, n+m)}$, crossing the top n edges with the remaining m edges. Note that n and m may be 0. Therefore, we derive the following equality of constructors:

$$\text{swap}(n, 0) = \text{swap}(0, n) = \text{edges}(n)$$

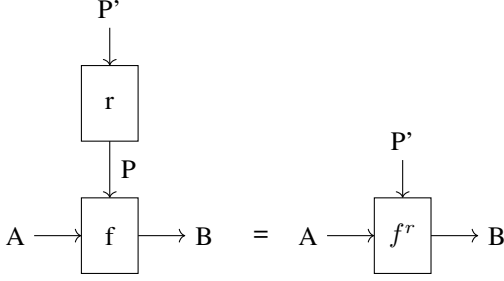


Figure 2. 2-Morphisms as reparameterizations

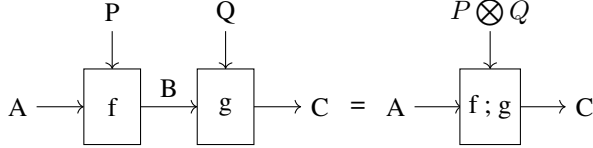


Figure 3. Sequential composition

This equality is already implicitly represented in our string diagram visualizations in Figure 1b and Figure 1c. It is possible to reparameterize a function. A reparameterization is the composition of a parameter function $r : P' \rightarrow P$ and a parametric function $f : A \bullet P \rightarrow B$, yielding a parametric function $f^r : A \bullet P' \rightarrow B$. This is depicted in Figure 2 as a string diagram, where the node r is applied from above and constructed by $\mathbf{repara}(r) : D_{(n,m)} \rightarrow D_{(n,m)}$. This is our first constructor, which constructs a new string diagram from a given one. Reading the equality in Figure 2 from left to right shows how to *compose* multiple string diagrams into one, abstracting away unnecessary details. Reading the equality in Figure 2 from right to left shows how to *decompose* a single string diagram into its hidden components. String diagrams allow us to work at the right level of abstraction visually and, e.g., implicitly handle the associativity of constructors, whereas the corresponding terms must always contain all information and therefore enforce term equations to hold. Besides reparameterization, we have two more ways to compose given string diagrams into new ones. Figure 3 shows a string diagram that sequentially composes two parametric functions $f : A \bullet P \rightarrow B$ and $g : B \bullet Q \rightarrow C$ to construct a parametric function $f;g : A \bullet (P \otimes Q) \rightarrow C$. The constructor is $\mathbf{before}(p \otimes q) : D_{(n,k)} \rightarrow D_{(k,m)} \rightarrow D_{(n,m)}$, indexed by the composed parameters $p \otimes q \in P \otimes Q$ of the resulting parametric function. $\mathbf{edges}(n)$ is the neutral element for the sequential composition of string diagrams of type $D_{(n,m)}$ or $D_{(m,n)}$.

Parametric functions $f : A \bullet P \rightarrow C$ and $g : B \bullet Q \rightarrow D$ can also be composed in parallel to construct a parametric function $f \parallel g : (A \odot B) \bullet (P \otimes Q) \rightarrow C \odot D$. The constructor is $\mathbf{beside}(p \otimes q) : D_{(n,m)} \rightarrow D_{(h,k)} \rightarrow D_{(n+h,m+k)}$, indexed over the composed parameters $p \otimes q \in P \otimes Q$ of the resulting parametric function. Parallel composition

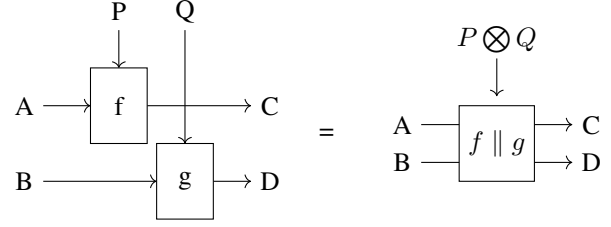


Figure 4. Parallel composition

```

before(beside( $f, g$ ), beside( $h, k$ ))
= beside(before( $f, h$ ), before( $g, k$ )); (interchange law)
before(beside(swap( $m, n$ ), edges( $p$ )), beside(edges( $n$ ), swap( $m, p$ )))
= swap( $m, n + p$ ); (swap law 1)
before(swap( $m, n$ ), swap( $n, m$ ))
= edges( $m + n$ ); (swap law 2)
before(beside(edges( $m$ ), swap( $n, p$ )), beside(swap( $m, p$ ), edges( $n$ )))
= swap( $m + n, p$ ); (swap law 3)
repara( $r$ , node( $f, p$ )) = node( $f$ ,  $r(p)$ ); (repara node)
...
    
```

Figure 5. Term equations

of string diagrams is shown in Figure 4. The empty string diagram is the neutral element for parallel composition.

Now that we have introduced all the constructors for string diagrams, we need to define the term equations that encode the categorical laws for the **Para**-construction. For brevity, we leave the definition of all necessary term equations to Appendix A and focus in Figure 5 only on those that weren't already mentioned and aren't as obvious as, e.g., associativity laws for constructors or the remaining repara laws. For simplicity, we omit the indices of the constructors where they are obvious from the context. The swap laws are derived from the assumption that the **Para**-construction is symmetric. The interchange law follows from the fact that the **Para**-construction is a monoidal category. While the associativity and neutrality laws are implicitly encoded in all string diagram representations, Figure 6 and Figure 7 provide a visual interpretation of the laws from Figure 5. Note that the string diagrams defined here do not follow the usual definition of string diagrams for 2-categories. This is due to the **Para**-construction's definition of 2-morphisms as reparameterizations.

Theorem 2.1. *The algebraic theory defined by the presented constructors and term equations is a **Para**-construction.*

Proof in Appendix A.

3. Combinatory Logic Search Space Synthesis

The full definition of FCLP is given in (Dudenhefner et al., 2023). Here, we define only the concepts necessary for Section 4.

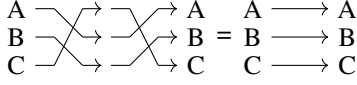


Figure 6. Symmetry law

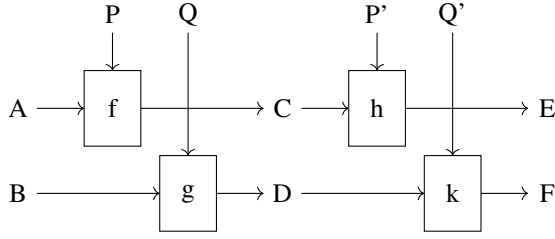


Figure 7. Interchange law

Definition 3.1 (Intersection Types with Covariant Constructors and Literals).

INTERSECTION TYPES

$$\mathbb{T} \ni \sigma, \tau, \rho ::= \omega \mid \sigma \rightarrow \tau \mid \sigma \cap \tau \mid C(\sigma) \mid l$$

where C ranges over an enumerable set of *unary type constructors*, and l ranges over an enumerable set of *literals*. For brevity, $C(\omega)$ is written C .

Definition 3.2 (Parameterized Types).

PARAMETERIZED TYPES

$$\varphi, \psi ::= \sigma \mid \langle y : t \rangle \Rightarrow \varphi \mid \langle x : \sigma \rangle \gg \Rightarrow \varphi \mid P \Rightarrow \varphi$$

where t ranges over collection identifiers and P ranges over decidable predicates, possibly containing literal variables and term variables, ranged over by x, y, z . A literal variable y is *bound* in $\langle y : t \rangle \Rightarrow \varphi$. A term variable x is *bound* in $\langle x : \sigma \rangle \gg \Rightarrow \varphi$.

For convenience, we extend the above definition of parameterized types with the following notational conventions:

$$\begin{aligned} &\langle x : t \rangle \Rightarrow \langle y : t \rangle \Rightarrow \varphi \\ &\Leftrightarrow \langle x, y : t \rangle \Rightarrow \varphi; \\ &\langle y : t_1 \rangle \Rightarrow \langle x : t_2 \rangle \Rightarrow (x = f(y)) \Rightarrow \varphi \\ &\Leftrightarrow \langle y : t_1 \rangle \Rightarrow \langle (x := f(y)) : t_2 \rangle \Rightarrow \varphi; \\ &\langle y : t_1 \rangle \Rightarrow \langle x : t_2 \rangle \Rightarrow \langle z : t_3 \rangle \Rightarrow (z = f(x, y)) \Rightarrow \varphi \\ &\Leftrightarrow \langle y : t_1 \rangle \Rightarrow \langle x : t_2 \rangle \Rightarrow \varphi[z := f(x, y)]; \end{aligned}$$

Definition 3.3 (Combinatory Terms and Arguments).

$$\begin{aligned} \text{COMBINATORY TERMS} \quad \mathbb{C} \ni M, N &::= A \mid M T \\ \text{COMBINATORY ARGUMENTS} \quad T &::= M \mid l \end{aligned}$$

where A, B, C range over an enumerable set of *combinators*.

A type environment, also called a repository, assigns parametric intersection types to combinators. A literal environment assigns names to collections of literals. Combinatory Logic Synthesis (CLS) is the process of solving the relativized type inhabitation problem for FCLP. The Combinatory Synthesizer (CoSy) (Dudenhefner et al., 2026; Stahl, 2025) is a component-oriented software synthesis framework that implements a type-inhabitation algorithm for FCLP.

Given a repository, a literal environment, and a parameterized intersection type, CoSy computes a *parameterized tree grammar* (Stahl, 2025) that exactly describes the tree language of combinatory terms inhabiting the requested type. We call such grammars *solution spaces*. Combinatory terms may be interpreted in any domain or programming language, provided each combinator’s interpretation adheres to the specification encoded in its parametric intersection type. For CLS, we therefore identify *components* as combinators and their *specifications* as parametric intersection types. CoSy provides methods for enumerating and interpreting terms from each computed solution space. However, this functionality alone is insufficient for applications that require optimizing a scoring function over combinatory terms. In these cases, the optimal term structure is usually unknown in advance. Our first contribution is a conservative extension of CoSy that equips synthesized solution spaces with sampling and search procedures. A *search space* is a solution space equipped with at least one sampling algorithm and search algorithms that are closed under the synthesized search space. The emptiness and finiteness problems for solution spaces are undecidable, yet semi-decidable (Dudenhefner et al., 2023), yielding an enumeration algorithm that is not guaranteed to terminate for every synthesized (possibly infinite) solution space. The same must be true for any sampling algorithm for search spaces. To address the feasibility of sampling, the following approach is proposed: For specifications that carefully use term predicates, the enumeration method can often decide the emptiness and finiteness problems. Sampling is performed under the user-guaranteed invariants that the search space is nonempty and large enough for the requested sample size. Users can test this invariant using the enumeration method. If enumeration does not produce a set of terms within a reasonable time, or if the search space is empty, both sampling and enumeration may fail to terminate. This is not considered a restriction, as generate-and-test remains feasible for small search spaces. Requiring users to verify the search space size first encourages consideration of generate-and-test methods when appropriate. The sampling method is a straightforward adaptation of a top-down sampling strategy from property-based testing (Hughes, 2007), with one modification: It will be shown later that a carefully defined specification can guarantee the finiteness of the search space. Accordingly, a Boolean flag is implemented to enable faster sampling

under the invariant that the search space is non-empty, finite, and contains a sufficient number of terms. Search methods closed under each synthesized search space are provided by adapting existing solutions for definite-clause grammars (Wong & Leung, 1997), and context-free grammars with constraints (Schrodi et al., 2023) to parameterized tree grammars. Two search methods are presented as prototypical examples. Evolutionary operations, specifically *crossover* and *mutation*, are defined on each synthesized search space by adapting algorithms from grammar-guided genetic programming on definite-clause grammars (Wong & Leung, 1997), that are closely related to parameterized tree grammars (Stahl, 2025). Because evolutionary algorithms require efficiently computable scoring functions, Bayesian optimization is proposed as a second search method for scoring functions that are computationally expensive. The approach from (Schrodi et al., 2023), which uses context-free grammars (CFGs) with constraints on the search space, can be adapted by replacing the CFGs with constraints and the evolutionary operators on them with parameterized tree grammars and the previously defined evolutionary operators. Given a kernel over combinatory terms, a Gaussian Process serves as a surrogate for the scoring function. The acquisition function is optimized using the evolutionary search method defined above. Recall that combinatory terms are language-agnostic and interpretable in any domain. This is particularly useful for graph kernels, as alternative graph interpretations allow the application of kernels without being restricted to the tree structure of combinatory terms. In Section 4, we will encode string diagrams that correspond to directed acyclic multigraphs (DAMG) as combinatory terms. Defining graph kernels based on the semantic DAMG structure and graph kernels on the tree structure of combinatory terms allows us to evaluate the practical relevance of our algebraic theory in Section 5.

Our extension of CoSy with evolutionary algorithms has already been adopted into the framework (Dudenhefner et al., 2026). Bayesian Optimization for CoSy is currently not part of the framework itself, but is part of the accompanying GitHub repository of elaborate examples (Laarmann, 2026a).

4. Modeling the Para-construction in FCLP

Our goal is to implement the algebraic theory from Section 2 as an FCLP repository to synthesize combinatory terms that represent string diagrams and to interpret them as parametric functions. The repository’s combinators correspond to the constructors of string diagrams. We explain the steps to get from the repository in Figure 8, which is a plain translation of the constructors from Section 2 into typed combinators, to the repository in Figure 9, which models the algebraic theory, including equations.

$$\begin{aligned} \Gamma = \{ \\ \text{node}(f, p) : < n, m : \mathbb{N} > \Rightarrow \\ & D(\text{IN}(n) \cap \text{OUT}(m)); \\ \text{edges} : < n : \mathbb{N} > \Rightarrow \\ & D(\text{IN}(n) \cap \text{OUT}(n)); \\ \text{swap} : < n, m : \mathbb{N} > \Rightarrow \\ & D(\text{IN}(n + m) \cap \text{OUT}(n + m)); \\ \text{repara}(r) : < n, m : \mathbb{N} > \Rightarrow \\ & D(\text{IN}(n) \cap \text{OUT}(m)) \rightarrow D(\text{IN}(n) \cap \text{OUT}(m)); \\ \text{before}(p \otimes q) : < n, m, k : \mathbb{N} > \Rightarrow D(\text{IN}(n) \cap \text{OUT}(k)) \\ & \rightarrow D(\text{IN}(k) \cap \text{OUT}(m)) \rightarrow D(\text{IN}(n) \cap \text{OUT}(m)); \\ \text{beside}(p \otimes q) : < n, m, h, k : \mathbb{N} > \Rightarrow D(\text{IN}(n) \cap \text{OUT}(m)) \\ & \rightarrow D(\text{IN}(h) \cap \text{OUT}(k)) \rightarrow D(\text{IN}(n + h) \cap \text{OUT}(m + k)); \\ & \epsilon : D(\text{IN}(0) \cap \text{OUT}(0)); \} \end{aligned}$$

Figure 8. String diagram constructors as an FCLP repository

The first point to note is that the signatures of indexed constructors already rely on the dependent type $D_{(n,m)}$ of string diagrams, indexed over their ingoing and outgoing edges, and are modeled as a literal abstraction over two literal variables $n, m \in \mathbb{N}$ in Figure 8. The second point is that we don’t have a product construction for (n, m) and therefore model it using intersection types by explicitly naming the projections IN and OUT, applying them as type constructors, and intersecting them. If we were to use the repository from Figure 8 for synthesis, we would compute search spaces with many string diagrams that are equivalent under the equations from the previous section, resulting in redundant interpretations of the combinatory terms. To overcome this, we will extend the parameterized intersection types to describe the quotient of combinatory terms under the equations of the algebraic theory. To do this, we utilize the approach from (Bessai et al., 2022), which restricts a tree grammar with a term rewriting system by forbidding its left-hand sides. Therefore, we direct the equations and describe how to synthesize only terms that match the right-hand sides of the resulting rules.

Lemma 4.1. *The resulting term rewriting system is terminating.*

Proof in Appendix A.

Hence every term rewrites to at least one normal form. Since the term rewriting system is not confluent (Proposition A.11), the normal forms cover the quotient but are not in bijection with it. To synthesize only terms matching the right-hand side of the associativity laws, we observe the list-like pattern of the terms on the right-hand side. Therefore, the proposed solution is a signature isomorphism that introduces two constructors for sequential

and parallel composition, each operating in a list-like way. To enforce the right-hand side of the interchange rule, we want to synthesize only terms that are sequential compositions of parallel compositions. This can be done by stratifying the type of string diagrams $D(\text{IN}(n) \cap \text{OUT}(m))$ into three types $D_0(\text{IN}(n) \cap \text{OUT}(m))$ for constructors **node**, **edges**, **swap**, $D_1(\text{IN}(n) \cap \text{OUT}(m))$ for string diagrams that are parallel compositions of these constructors, and $D_2(\text{IN}(n) \cap \text{OUT}(m))$ for string diagrams that are sequential compositions of parallel compositions. The right-hand sides of the repara laws show that there is no normal form that contains a **repara**-constructor. Therefore, we must remove the **repara**-constructor from the repository. This makes sense, as combinators are indexed by their parameters, and reparameterizations are expressed by using the same combinator with a different argument. We must use the **edges**(n) constructor rather than n parallel compositions of **edges**(1) or any combination of different numbers of edges. Additionally, because edges are neutral elements under sequential composition, we need to track whether a parallel composition consists solely of edges. This is encoded as an intersection type that uses the types ID , $\overline{\text{ID}}$, $\text{LAST}(\text{ID})$, $\text{LAST}(\overline{\text{ID}})$ as labels, indicating whether a parallel composition is an identity (ID , $\overline{\text{ID}}$) and whether it is allowed to compose another **edges** constructor to ensure correct accumulation of **edges** under one constructor ($\text{LAST}(\text{ID})$, $\text{LAST}(\overline{\text{ID}})$). Since a **swap**(n, m) constructor with $n \neq 0$ and $m \neq 0$ is not an identity, whereas **edges**(n) is, we distinguish the two constructors and enforce a partition between them using predicates P_{edges} and P_{swap} . The swap laws are challenging to encode using intersection types because they involve a deep pattern match on the constructor arguments. As a result, the left-hand sides of these laws are forbidden by term-predicates.

The goal of this paper is to define a general framework for synthesizing search spaces for parametric functions. The authors therefore argue that synthesizing a search space in which the empty string diagram is a valid solution is never intended. The proposed repository in Figure 9 will therefore explicitly exclude the construction of the empty string diagram and all corresponding laws. Some constructors in Figure 8 are still indexed; e.g., the **node** constructor is indexed by its function identifier and its parameters. We will introduce literal variables for these indices as well, and refine the intersection type to also encode this information. In Figure 9, all information the **node** constructor abstracts over is encoded in an extra literal variable s and therefore is encoded in its parametric intersection type under the type constructor **STRUCT**. All constraints on s are handled by the predicate P_{node} . We will distinguish three collections of literals for this encoding. The collection $\mathbb{L}_0$ will consist of triples, where the first projection identifies the concrete parametric function (the function identifier and parameters for **node**, special identifiers for **edges** and **swap**), and the sec-

ond and third projections give the input and output numbers of edges in the constructed string diagram. The collection $\mathbb{L}_1$ consists of sequences of elements in $\mathbb{L}_0$ and serves as an encoding for parallel composition. The collection $\mathbb{L}_2$ will consist of sequences of elements in $\mathbb{L}_1$ and will be used to encode sequential composition. This encoding scheme for all relevant string diagram information follows the composition structure and allows us to define parametric intersection types that precisely specify string diagrams in normal form. The consistency of this encoding, as well as the swap laws, is handled with predicates $P_{\text{beside.s}}$, $P_{\text{beside.c}}$, $P_{\text{before.s}}$, and $P_{\text{before.c}}$. With an additional predicate that determines whether two encodings are in the same equivalence class, one can even synthesize a normal form for encodings that are not in normal form and ensure that, for any encoding in the codomain, only the normal forms of that encoding will be requested in the domain.

Theorem 4.2 (Uniqueness of encoding). *Any structure encoding of an inhabitant of the repository in Figure 9 is isomorphic to its corresponding string diagram.*

Proof in Appendix A.

We can now provide a unique intersection type for each synthesized string diagram. To precisely control variance in the synthesized search space, we introduce a special literal ζ that encodes any valid structure at this position in the string diagram. We allow ζ to appear in literals from $\mathbb{L}_0$ and $\mathbb{L}_1$, but not $\mathbb{L}_2$, forcing the structure of any sequential composition to be of a fixed length. We propagate each ζ from the requested type through the compositional structure to the combinators that inhabit ζ as well as to a specific literal, thereby obtaining a precise way to describe the variance of string diagrams in the search space. One may choose to allow ζ for $\mathbb{L}_2$ as well, but this will yield an infinite search space. As mentioned in Section 3, we aim to model a finite search space to obtain a faster sampling algorithm.

Lemma 4.3. *The repository in Figure 9 specifies only finite search spaces if $\mathbb{L}_0$ and $\mathbb{L}_1$ are finite.*

Lemma 4.3 is a direct consequence of the previous argument. Because $\mathbb{L}_2$ may be infinite, we require each type to specify a finite sequence $s \in \mathbb{L}_2$. It doesn't matter how often ζ occurs in s , the number of possible sequences obtained by unfolding ζ in s is finite, since the numbers of possible elements in $\mathbb{L}_0$ and $\mathbb{L}_1$ are finite. To reduce complexity and keep the definitions accessible, we restrict the encoding of variance in Figure 9 to elements of $\mathbb{L}_0$ and $\mathbb{L}_1$, as well as the numbers of ingoing and outgoing edges. Further variance can be encoded using the same principles. To conclude this section, we will discuss an example.

For readability, we write $(;)$ and $(||)$ as infix constructors for **before** and **beside** in the following definition of term t that represents the string diagram in Figure 10. The term t is the only inhabitant of the following intersection

$\Gamma = \{$
node : $\langle f : \mathbb{F} \rangle \Rightarrow \langle p : \mathbb{P}_f \rangle \Rightarrow \langle n, m : \mathbb{N} \rangle \Rightarrow$
 $\langle ((s := ((f, p), n, m)) : \mathbb{L}_0) \Rightarrow P_{\text{node}}(s) \Rightarrow$
 $D_0(\text{IN}(n) \cap \text{IN}(\dot{i}) \cap \text{OUT}(m) \cap \text{OUT}(\dot{i})) \cap$
 $\text{STRUCT}(s) \cap \text{STRUCT}(\dot{i}) \cap \overline{\text{ID}};$
edges : $\langle n : \mathbb{N} \rangle \Rightarrow$
 $\langle ((s := ((\text{swap}, n, 0), n, n)) : \mathbb{L}_0) \Rightarrow$
 $\Rightarrow P_{\text{edges}}(s) \Rightarrow$
 $D_0(\text{IN}(n) \cap \text{IN}(\dot{i}) \cap \text{OUT}(n) \cap \text{OUT}(\dot{i})) \cap$
 $\text{STRUCT}(s) \cap \text{STRUCT}(\dot{i}) \cap \text{ID};$
swap : $\langle n, m : \mathbb{N} \rangle \Rightarrow$
 $\langle ((s := ((\text{swap}, n, m), n + m, n + m)) : \mathbb{L}_0) \Rightarrow$
 $P_{\text{swap}}(s) \Rightarrow$
 $D_0(\text{IN}(n + m) \cap \text{IN}(\dot{i}) \cap \text{OUT}(n + m) \cap \text{OUT}(\dot{i})) \cap$
 $\text{STRUCT}(s) \cap \text{STRUCT}(\dot{i}) \cap \overline{\text{ID}};$
beside.s : $\langle n, m : \mathbb{N} \rangle \Rightarrow \langle p : \mathbb{L}_0 \rangle \Rightarrow$
 $\langle (q := [p] : \mathbb{L}_1) \Rightarrow P_{\text{beside.s}}(n, m, p, q) \Rightarrow$
 $((D_0(\text{IN}(n) \cap \text{OUT}(m) \cap \text{STRUCT}(p)) \cap \overline{\text{ID}}) \rightarrow$
 $(D_1(\text{IN}(n) \cap \text{OUT}(m) \cap \text{STRUCT}(q) \cap$
 $\text{STRUCT}(\dot{i}) \cap \overline{\text{ID}} \cap \text{LAST}(\overline{\text{ID}}))) \cap$
 $((D_0(\text{IN}(n) \cap \text{OUT}(m) \cap \text{STRUCT}(p)) \cap \text{ID}) \rightarrow$
 $(D_1(\text{IN}(n) \cap \text{IN}(\dot{i}) \cap \text{OUT}(m) \cap \text{OUT}(\dot{i})) \cap$
 $\text{STRUCT}(q) \cap \text{STRUCT}(\dot{i}) \cap \text{ID}));$
beside.c : $\langle n, m, h, k : \mathbb{N} \rangle \Rightarrow \langle p : \mathbb{L}_0 \rangle \Rightarrow$
 $\langle q : \mathbb{L}_1 \rangle \Rightarrow P_{\text{beside.c}}(n, m, h, k, p, q) \Rightarrow$
 $((D_0(\text{IN}(n) \cap \text{OUT}(m) \cap \text{STRUCT}(p)) \cap \text{ID}) \rightarrow$
 $(D_1(\text{IN}(h) \cap \text{OUT}(k) \cap \text{STRUCT}(q)) \cap$
 $\overline{\text{ID}} \cap \text{LAST}(\overline{\text{ID}})) \rightarrow$
 $(D_1(\text{IN}(n + h) \cap \text{IN}(\dot{i}) \cap \text{OUT}(m + k) \cap \text{OUT}(\dot{i})) \cap$
 $\text{STRUCT}([p] + q) \cap \text{STRUCT}(\dot{i}) \cap \overline{\text{ID}} \cap \text{LAST}(\text{ID}))) \cap$
 $((D_0(\text{IN}(n) \cap \text{OUT}(m) \cap \text{STRUCT}(p)) \cap \overline{\text{ID}}) \rightarrow$
 $(D_1(\text{IN}(h) \cap \text{OUT}(k) \cap \text{STRUCT}(q)) \cap \overline{\text{ID}}) \rightarrow$
 $(D_1(\text{IN}(n + h) \cap \text{IN}(\dot{i}) \cap \text{OUT}(m + k) \cap \text{OUT}(\dot{i})) \cap$
 $\text{STRUCT}([p] + q) \cap \text{STRUCT}(\dot{i}) \cap \overline{\text{ID}} \cap \text{LAST}(\overline{\text{ID}}))) \cap$
 $((D_0(\text{IN}(n) \cap \text{OUT}(m) \cap \text{STRUCT}(p)) \cap \overline{\text{ID}}) \rightarrow$
 $(D_1(\text{IN}(h) \cap \text{OUT}(k) \cap \text{STRUCT}(q)) \cap \overline{\text{ID}}) \rightarrow$
 $(D_1(\text{IN}(n + h) \cap \text{IN}(\dot{i}) \cap \text{OUT}(m + k) \cap \text{OUT}(\dot{i})) \cap$
 $\text{STRUCT}([p] + q) \cap \text{STRUCT}(\dot{i}) \cap \overline{\text{ID}} \cap \text{LAST}(\overline{\text{ID}})));$
before.s : $\langle n, m : \mathbb{N} \rangle \Rightarrow \langle p : \mathbb{L}_1 \rangle \Rightarrow \langle (q := [p] : \mathbb{L}_2) \Rightarrow$
 $\langle x : D_1(\text{IN}(n) \cap \text{OUT}(m) \cap \text{STRUCT}(p)) \rangle \Rightarrow$
 $P_{\text{before.s}}(n, m, p, q, x) \Rightarrow D_2(\text{IN}(n) \cap \text{IN}(\dot{i}) \cap$
 $\text{OUT}(m) \cap \text{OUT}(\dot{i}) \cap \text{STRUCT}(q));$
before.c : $\langle n, m, k : \mathbb{N} \rangle \Rightarrow \langle a : \mathbb{L}_2 \rangle \Rightarrow$
 $\langle (b := \text{normalize}(a) : \mathbb{L}_2) \rangle \Rightarrow$
 $\langle (p := \text{head}(b) : \mathbb{L}_1) \rangle \Rightarrow \langle (q := \text{tail}(b) : \mathbb{L}_2) \rangle \Rightarrow$
 $\langle x : D_1(\text{IN}(n) \cap \text{OUT}(k) \cap \text{STRUCT}(p)) \cap \overline{\text{ID}} \rangle \Rightarrow$
 $\langle y : D_2(\text{IN}(k) \cap \text{OUT}(m) \cap \text{STRUCT}(q)) \rangle \Rightarrow$
 $P_{\text{before.c}}(n, m, p, q, x, y) \Rightarrow$
 $D_2(\text{IN}(n) \cap \text{IN}(\dot{i}) \cap \text{OUT}(m) \cap \text{OUT}(\dot{i}) \cap \text{STRUCT}(a))$
 $\}$

Figure 9. The repository for our algebraic theory

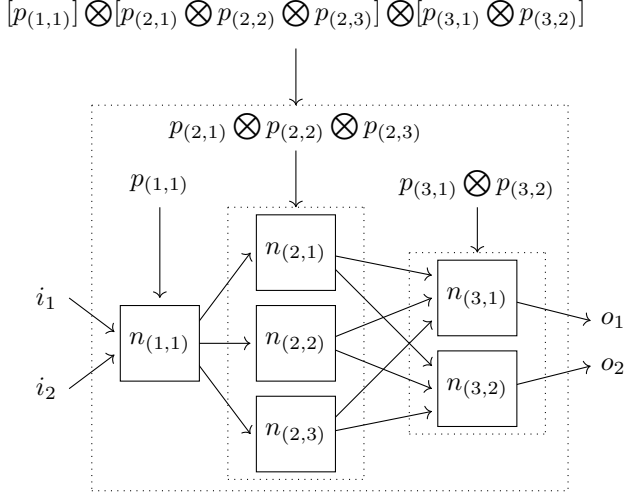


Figure 10. Neural network as string diagram

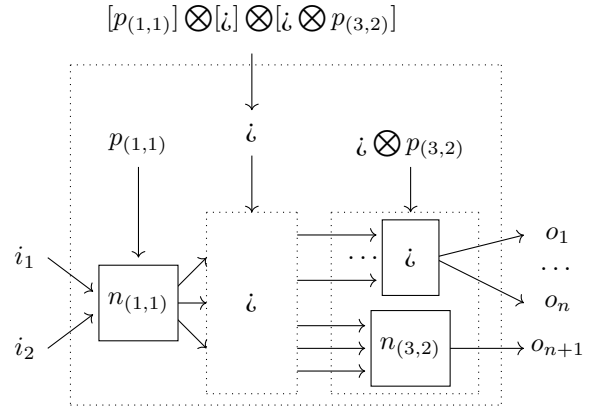


Figure 11. Structural variance in synthesized string diagrams

type σ and can also be assigned the super type σ' of σ .

$t := \text{node}(n_{(1,1)}, p_{(1,1)}) ;$
 $(\text{node}(n_{(2,1)}, p_{(2,1)}) \parallel \text{node}(n_{(2,2)}, p_{(2,2)}) \parallel$
 $\text{node}(n_{(2,3)}, p_{(2,3)})) ;$
 $(\text{edges}(1) \parallel \text{swap}(1, 1) \parallel \text{swap}(1, 1) \parallel \text{edges}(1)) ;$
 $(\text{edges}(2) \parallel \text{swap}(1, 1) \parallel \text{edges}(2)) ;$
 $(\text{node}(n_{(3,1)}, p_{(3,1)}) \parallel \text{node}(n_{(3,2)}, p_{(3,2)}))$
 $\sigma := D_2(\text{IN}(2) \cap \text{OUT}(2) \cap \text{STRUCT}([((n_{(1,1)}, p_{(1,1)}), 2, 3)],$
 $[((n_{(2,1)}, p_{(2,1)}), 1, 2), ((n_{(2,2)}, p_{(2,2)}), 1, 2),$
 $((n_{(2,3)}, p_{(2,3)}), 1, 2)],$
 $[((\text{swap}, 1, 0), 1, 1), ((\text{swap}, 1, 1), 2, 2),$
 $((\text{swap}, 1, 1), 2, 2), ((\text{swap}, 1, 0), 1, 1)],$
 $[((\text{swap}, 2, 0), 2, 2), ((\text{swap}, 1, 1), 2, 2),$
 $((\text{swap}, 2, 0), 2, 2)],$
 $[((n_{(3,1)}, p_{(3,1)}), 3, 1), ((n_{(3,2)}, p_{(3,2)}), 3, 1)]])$

$$\begin{aligned} \sigma' := & D_2(\text{IN}(2) \cap \text{OUT}(\dot{i}) \cap \text{STRUCT}([\\ & [(n_{(1,1)}, p_{(1,1)}), 2, 3], \\ & \dot{i}, \\ & [(swap, 1, 0), 1, 1), ((swap, 1, 1), 2, 2), \\ & ((swap, 1, 1), 2, 2), ((swap, 1, 0), 1, 1)], \\ & [(swap, 2, 0), 2, 2), ((swap, 1, 1), 2, 2), \\ & ((swap, 2, 0), 2, 2)], \\ & [\dot{i}, ((n_{(3,2)}, p_{(3,2)}), 3, 1)]]) \end{aligned}$$

Therefore, the variance of string diagrams within a search space relies on the possibility of assigning more than one type to a term (see (Dudenhefner et al., 2023)). The intersection type σ' describes a search space with multiple inhabitants and can also be depicted as a string diagram with holes, as seen in Figure 11.

5. Proof of Concept

We now evaluate the practical consequences of synthesizing search spaces using our algebraic theory for parametric functions. The experiments presented in this section are available through a GitHub repository (Laarmann, 2026b). The goal of this section is not to introduce a new optimization method, but to investigate how synthesized search spaces interact with standard optimization techniques such as kernel-based surrogate modeling and Bayesian optimization. To do so, we consider a simple one-dimensional process inspired by a physical bounce or reflection: an object moves linearly towards a boundary, changes direction at the boundary, and then moves linearly away from it. This gives rise to the piecewise-affine target function

$$f^*(x) = \begin{cases} x + 10, & x \leq 0, \\ 10 - x, & x > 0. \end{cases}$$

Equivalently, on the considered domain, this corresponds to the function $10 - |x|$, which exhibits a sharp change in direction at $x = 0$. The relevance of this example lies not in the difficulty of the regression problem but in its compositional structure: the target mechanism consists of a splitting decision followed by two affine branches. For evaluation, we generate training samples by evaluating f^* on $N_{\text{train}} = 1000$ points from the interval $[-10, 10]$, adding small Gaussian noise $\varepsilon \sim \mathcal{N}(0, 0.05)$ to account for moderate observation noise. To evaluate whether a learned model captures the underlying mechanism rather than merely interpolating the observed samples, we evaluate the model on $N_{\text{test}} = 1000$ points from the larger interval $[-15, 15]$. The test interval, therefore, contains out-of-distribution regions not observed during training. To synthesize search spaces from the repository in Figure 9, we first need to define the components from which we will construct parametric functions. As components, we want to

use a selection of PyTorch modules, which are embedded in the repository by providing the literal collections $\mathbb{F}$ and $\mathbb{P}_f$ to define the literal collection $\mathbb{L}_0$, uniquely identifying each valid node. Therefore, the identifiers in $\mathbb{F} = \{\text{Linear}, \text{Sigmoid}, \text{ReLU}, \text{Tanh}, \text{Sum}, \text{Product}, \text{Copy}\}$ correspond to PyTorch module names and tensor operations. $\mathbb{P}_f$ then contains valid hyperparameters for each module identifier $f \in \mathbb{F}$ that comply with the PyTorch module parameters. Consequently, any synthesized term can be straightforwardly interpreted as a PyTorch module by pattern matching on the fifth literal argument from $\mathbb{L}_0$ of the node constructor. The other constructors from the repository are implemented as the corresponding gating, branching, and composition operators from PyTorch. For our experiment, we set the literal collection $\mathbb{N} = \{1, 2, 3, 4, 5\}$ and consider various search spaces, all uniquely defined by an intersection type τ to control their variance and restrict their size.

$$\begin{aligned} \tau_3 := & D_2(\text{IN}(1) \cap \text{OUT}(1) \cap \text{STRUCT}([\dot{i}, \dot{i}, \dot{i}])) \\ \tau_{3\text{-refined}} := & D_2(\text{IN}(1) \cap \text{OUT}(1) \cap \text{STRUCT}([\dot{i}, \dot{i}, [\dot{i}]])) \\ & \dots \\ \tau_{5\text{-refined-2}} := & D_2(\text{IN}(1) \cap \text{OUT}(1) \cap \text{STRUCT}([\dot{i}, [\dot{i}, \dot{i}, \dot{i}], \dot{i}, [\dot{i}, \dot{i}], [\dot{i}]])) \end{aligned}$$

Each term is interpreted as a PyTorch module implementing the parametric function, which is trained on the training data for 2000 epochs, and its loss is computed on the test data. Therefore, our objective function f_{obj} is the interpretation of the term as the test loss of the parametric function, which corresponds to the term, compared to f^* . We use a Gaussian process regressor as the surrogate model. The kernel determines the surrogate’s capacity to approximate the objective function. We analyze how kernel choice influences model fidelity. For each synthesized search space, we sample 100 terms and quantify kernel-objective alignment by the Spearman rank correlation between pairwise kernel similarities and negative objective differences,

$$\rho(k(x_i, x_j), -|f_{\text{obj}}(x_i) - f_{\text{obj}}(x_j)|),$$

so that higher values indicate stronger agreement between kernel similarity and objective proximity. To reduce noise, the kernel-objective alignment is computed using the mean objective value across multiple training seeds. We compare three kernel constructions: a subtree kernel applied to the combinatory terms, a Weisfeiler-Lehman kernel applied to the same terms, and a Weisfeiler-Lehman kernel applied to the DAMG obtained by interpreting a term as a string diagram. The latter exploits the categorical semantics of the synthesized models by operating on the structural representation induced by the Para-construction. Across all considered search spaces, kernels operating on DAMGs consistently exhibit stronger alignment with the objective than kernels applied directly to the syntactic tree representation of the combinatory terms, as can be seen in Figure 12.

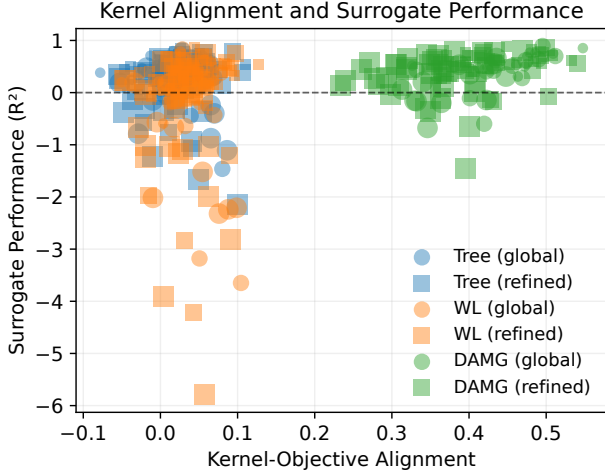


Figure 12. Kernel-objective alignment vs. surrogate learning performance

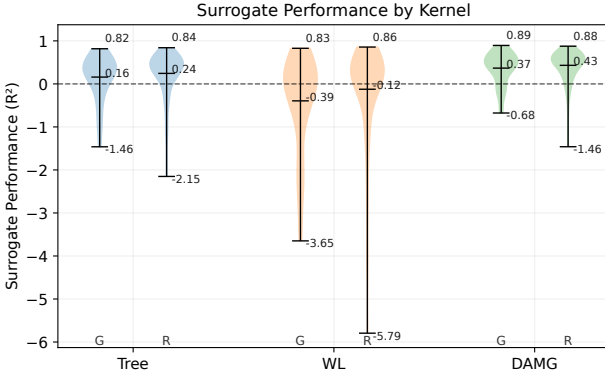


Figure 13. Surrogate learning performance across kernels. G: maximal variance; R: restricted variance

This suggests that the structural information captured by the categorical string diagram representation is more closely related to the functional behavior of the synthesized models than the purely syntactic tree structure of the combinatory terms. We next evaluate whether the objective function can be effectively approximated by surrogate models defined over the synthesized search spaces. For each kernel, we train a Gaussian process regressor on sampled candidates and evaluate predictive performance using normalized mean squared error and the R^2 score. Gaussian processes using the DAMG kernel consistently achieve lower prediction error than the corresponding tree-based kernels, see Table 1. Furthermore, the refined search spaces tend to exhibit improved predictive accuracy compared to their less-restricted counterparts, as shown in Figure 13. This indicates that restricting structural variance can improve the learnability of the objective over the synthesized search space. In Appendix B, we evaluate Bayesian optimization on the synthesized search spaces and compare it to random search under

Kernel	Space	NMSE $\downarrow$		$R^2 \uparrow$	
		Mean	Max	Mean	Max
Tree	Global	0.837	<u>2.752</u>	0.158	0.817
	Refined	0.946	4.559	0.244	0.841
WL	Global	1.035	3.032	-0.394	0.827
	Refined	1.046	4.845	-0.124	0.856
DAMG	Global	0.781	2.921	0.367	<u>0.893</u>
	Refined	0.874	4.664	0.431	0.876

Table 1. Mean and max NMSE ($\downarrow$) and R^2 ($\uparrow$) for Tree, WL, and DAMG kernels. Best mean values in bold; best maxima underlined

the same evaluation budget. In all configurations, Bayesian optimization identifies better candidates than random search within the given budget. At the same time, the optimization performance varies across the considered specifications: certain refinements improve optimization performance, while others restrict the search space in ways that make the optimization problem more difficult. The best candidate found across all experiments for f_{obj} exhibits an unconventional model structure. We discuss it in Appendix C. Overall, these experiments demonstrate that synthesized search spaces can be explored effectively using standard optimization techniques. Moreover, they indicate that the categorical structure underlying the string diagram representation provides a description well-suited to kernel-based surrogate modeling. These results support the view that categorical structure provides a useful inductive bias for defining and exploring search spaces of parametric functions.

6. Conclusion

This paper introduces a framework for synthesizing search spaces of parametric functions from typed component specifications. We provide an FCLP repository encoding the algebraic theory of parametric functions induced by the Para-construction and extend the CoSy framework so that synthesized solution spaces can serve as search spaces equipped with sampling and optimization procedures. The framework enables the automatic construction of search spaces from component specifications rather than requiring manually designed architecture grammars. Our proof-of-concept implementation demonstrates that the synthesized spaces can be explored using standard optimization techniques such as kernel-based surrogate modeling and Bayesian optimization. Our experiments indicate that the string diagram representation induced by the categorical model provides structural information that is beneficial for kernel-based modeling of the objective function. From a theoretical perspective, an important topic for future work is categorical semantics for FCLP allowing formal reasoning about soundness and completeness of models of parametric functions.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

- Bessai, J., Czajka, L., Laarmann, F., and Rehof, J. Restricting tree grammars with term rewriting. In Felty, A. P. (ed.), *7th International Conference on Formal Structures for Computation and Deduction, FSCD 2022, August 2-5, 2022, Haifa, Israel*, volume 228 of *LIPICs*, pp. 14:1–14:19. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022. doi: 10.4230/LIPICs.FSCD.2022.14. URL <https://doi.org/10.4230/LIPICs.FSCD.2022.14>.
- Capucci, M., Gavranović, B., Hedges, J., and Rischel, E. Towards foundations of categorical cybernetics. *Electronic Proceedings in Theoretical Computer Science*, 372: 235–248, 11 2022. doi: 10.4204/EPTCS.372.17.
- Cruttwell, G. S. H., Gavranović, B., Ghani, N., Wilson, P., and Zanasi, F. Categorical foundations of gradient-based learning. In Sergey, I. (ed.), *Programming Languages and Systems*, pp. 1–28, Cham, 2022. Springer International Publishing. ISBN 978-3-030-99336-8.
- David, C. and Kroening, D. Program synthesis: challenges and opportunities. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 375(2104):20150403, 2017.
- Dudenhefner, A., Stahl, C., Chaumet, C., Laarmann, F., and Rehof, J. Finite Combinatory Logic with Predicates. In Reyes, E. H., van den Berg, B., and Kesner, D. (eds.), *29th International Conference on Types for Proofs and Programs (TYPES 2023)*, Dagstuhl, Germany, 2023. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- Dudenhefner, A., Laarmann, F., Chaumet, C., and Stahl, C. combinatory-synthesizer. <https://github.com/tudo-seal/cosy>, 2026.
- Ericsson, L., Espinosa, M., Yang, C., Antoniou, A., Storkey, A., Cohen, S. B., McDonagh, S., and Crowley, E. J. einspace: Searching for neural architectures from fundamental operations. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL <https://openreview.net/forum?id=qflncViBr5>.
- Fong, B., Spivak, D., and Tuyéras, R. Backprop as functor: A compositional perspective on supervised learning. In *2019 34th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*, pp. 1–13. IEEE, 2019.
- Gavranović, B. *Fundamental Components of Deep Learning: A category-theoretic approach*. Phd thesis, University of Strathclyde, 2024. URL <https://doi.org/10.48550/arXiv.2403.13001>.
- Gavranović, B., Lessard, P., Dudzik, A., Von Glehn, T., Araújo, J. a. G., and Veličković, P. Position: categorical deep learning is an algebraic theory of all architectures. In *Proceedings of the 41st International Conference on Machine Learning, ICML’24*. JMLR.org, 2024.
- Gibbons, J. An initial-algebra approach to directed acyclic graphs. In Möller, B. (ed.), *Mathematics of Program Construction*, pp. 282–303, Berlin, Heidelberg, 1995. Springer Berlin Heidelberg. ISBN 978-3-540-49445-4.
- Hughes, J. Quickcheck testing for fun and profit. In *Proceedings of the 9th International Conference on Practical Aspects of Declarative Languages, PADL’07*, pp. 1–32, Berlin, Heidelberg, 2007. Springer-Verlag. ISBN 3540696083. doi: 10.1007/978-3-540-69611-7_1. URL https://doi.org/10.1007/978-3-540-69611-7_1.
- Jia, Y., Peng, G., Yang, Z., and Chen, T. Category-theoretical and topos-theoretical frameworks in machine learning: A survey. *Axioms*, 14:204, 03 2025. doi: 10.3390/axioms14030204.
- Laarmann, F. Elaborate examples for the CoSy-framework. <https://github.com/tudo-seal/cosy-examples>, 2026a.
- Laarmann, F. Experiments for the ICML 2026 Paper Search Space Synthesis for Parametric Functions. <https://github.com/tudo-seal/ICML2026-CoSy-Experiments>, 2026b.
- Liang, P., Jordan, M. I., and Klein, D. Learning programs: a hierarchical bayesian approach. In *Proceedings of the 27th International Conference on International Conference on Machine Learning, ICML’10*, pp. 639–646, Madison, WI, USA, 2010. Omnipress. ISBN 9781605589077.
- Mac Lane, S. *Categories for the working mathematician*, volume 5. Springer Science & Business Media, 1998.
- Negrinho, R., Gormley, M., Gordon, G. J., Patil, D., Le, N., and Ferreira, D. Towards modular and programmable architecture search. In Wallach, H., Larochelle, H., Beygelzimer, A., d’Alché-Buc, F., Fox, E., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019.
- Rehof, J. and Vardi, M. Y. Design and synthesis from components (dagstuhl seminar 14232). *Dagstuhl Reports*, 4(6):29–47, 2014.

Schrodi, S., Stoll, D., Ru, B., Sukthanker, R., Brox, T., and Hutter, F. Construction of hierarchical neural architecture search spaces based on context-free grammars. *Advances in Neural Information Processing Systems*, 36:23172–23223, 2023.

Stahl, C. *Component-Based Synthesis using Combinatory Logic, Intersection Types and Predicates*. Phd thesis, TU Dortmund University, 2025. URL <http://dx.doi.org/10.17877/DE290R-25780>.

Wong, M. L. and Leung, K. S. Evolutionary program induction directed by logic grammars. *Evol. Comput.*, 5(2): 143–180, June 1997. ISSN 1063-6560. doi: 10.1162/evco.1997.5.2.143. URL <https://doi.org/10.1162/evco.1997.5.2.143>.

A. Category-Theoretic Background and Proofs

This appendix provides the categorical background and formal proofs underlying the algebraic theory of parametric functions used in the main text.

We assume a basic knowledge of category theory for this section, or recommend reading (Mac Lane, 1998).

A.1. The Para-construction

We restrict attention to strict monoidal categories, which allow us to express natural isomorphisms as equations.

Definition A.1 ($\mathcal{M}$ -Actegory (compare (Gavranović, 2024))). Let $(\mathcal{M}, \otimes, I)$ be a strict monoidal category. A strict $(\mathcal{M}, \otimes, I)$ -actegory is a category $\mathcal{C}$ together with a functor $\bullet : \mathcal{C} \times \mathcal{M} \rightarrow \mathcal{C}$, and two natural isomorphisms called *the unitor* $\eta_C : C \bullet I = C$ and *the multiplier* $\mu_{C,M,N} : C \bullet (M \otimes N) = (C \bullet M) \bullet N$ whose components are objects $C : \mathcal{C}$ and $M, N : \mathcal{M}$, satisfying the following coherence laws:

- Pentagon law: For all $C : \mathcal{C}$ and $M, N, P : \mathcal{M}$, $C \bullet (M \otimes (N \otimes P)) = ((C \bullet M) \bullet N) \bullet P$.
- Unit laws: For all $C : \mathcal{C}$ and $M : \mathcal{M}$, $C \bullet (M \otimes I) = C \bullet M$ and $C \bullet (I \otimes M) = C \bullet M$.

Definition A.2 (Para-construction (compare (Gavranović, 2024))). Let $(\mathcal{M}, \otimes, I)$ be a strict monoidal category and $(\mathcal{C}, \bullet)$ be a strict $(\mathcal{M}, \otimes, I)$ -actegory. We define the bicategory $\mathbf{Para}_\bullet(\mathcal{C})$ of parametric morphisms as follows:

- The objects are those of $\mathcal{C}$;
- The morphisms of type $A \rightarrow B$ are pairs (P, f) for parameter spaces $P : \mathcal{M}$, morphisms $f : A \bullet P \rightarrow B$, and objects $A, B : \mathcal{C}$;
- The 2-morphisms of type $(P, f) \Rightarrow (P', f')$ are morphisms $r : P' \rightarrow P$ in $\mathcal{M}$ such that the following diagram

commutes:

$$\begin{array}{ccc} A \bullet P' & \xrightarrow{f'} & B \\ \downarrow A \bullet r & \nearrow f & \\ A \bullet P & & \end{array}$$

- The identities are morphisms (I, η_A) for every object $A : \mathcal{C}$;
- The sequential composition of morphisms (P, f) and (Q, g) yields a morphism $(P \otimes Q, f; g)$ where $(f; g) : A \bullet (P \otimes Q) \rightarrow C$ in $\mathcal{C}$.
- Horizontal and vertical composition of 2-morphisms are given by the monoidal structure of $\mathcal{M}$.

Definition A.3 (Symmetric Strict Monoidal Actegory (compare (Gavranović, 2024))). Let $(\mathcal{M}, \otimes, I)$ be a symmetric strict monoidal category. An $\mathcal{M}$ -actegory $(\mathcal{C}, \bullet)$ is called a symmetric strict monoidal $\mathcal{M}$ -actegory if the underlying category $\mathcal{C}$ has a symmetric monoidal structure $(\odot, J)$ and the parameterization operation $\bullet$ respects that structure. This means that

- The underlying functor $\mathcal{C} \times \mathcal{M} \rightarrow \mathcal{C}$ is strong monoidal;
- The underlying natural isomorphisms η and μ are monoidal;

Definition A.4 (Symmetric Strict Monoidal Para-construction (compare (Gavranović, 2024))). Let $(\mathcal{C}, \bullet)$ be a symmetric strict monoidal $\mathcal{M}$ -actegory with the underlying symmetric monoidal structure $(\odot, J)$. Then $\mathbf{Para}_\bullet(\mathcal{C})$ becomes a symmetric strict monoidal bicategory with the following constructions additionally to Definition A.2:

- The monoidal product of any two objects $A, B : \mathcal{C}$ is given by $A \odot B$ of $(\mathcal{C}, \odot, J)$;
- The monoidal unit is the monoidal unit of $(\mathcal{C}, \odot, J)$;
- The monoidal product of two morphisms $(P, f : A \bullet P \rightarrow B)$ and $(Q, g : C \bullet Q \rightarrow D)$ is given by $(P \otimes Q, f \odot g : (A \odot C) \bullet (P \otimes Q) \rightarrow B \odot D)$.

- The braiding operation $\beta_{A,B} : A \odot B \rightarrow B \odot A$;
- The symmetry law $\beta_{A,B}; \beta_{B,A} = (I, \eta_A \odot_B)$ must hold;
- The interchange law $(P, f) \odot (P', h); ((Q, g) \odot (Q', i)) = ((P, f); (Q, g)) \odot ((P', h); (Q', i))$ must hold;

A.2. Algebraic Theory

Let us summarize the results of Section 2 as follows:

Definition A.5 (Signature).

$$\Sigma = \{$$

$$\begin{aligned} &\mathbf{node}(n, m, f, p) : D_{(n,m)} \\ &\mathbf{edges}(n) : D_{(n,n)} \\ &\mathbf{swap}(n, m) : D_{(n+m,n+m)} \\ &\mathbf{repara}(n, m, r) : D_{(n,m)} \rightarrow D_{(n,m)} \\ &\mathbf{before}(n, k, m, p \otimes q) : D_{(n,k)} \rightarrow D_{(k,m)} \rightarrow D_{(n,m)} \\ &\mathbf{beside}(n, m, h, k, p \otimes q) : D_{(n,m)} \rightarrow D_{(h,k)} \rightarrow D_{(n+h,m+k)} \\ &\epsilon : D_{(0,0)} \end{aligned}$$

$$\}$$

We define term variables $x_{(n,m)} : D_{(n,m)} \in V$ indexed over the incoming and outgoing edges to enforce type correctness for the indexed sorts $D_{(n,m)}$. We omit the indices, if they are clear from the context.

Definition A.6 (Term Equations).

$$\begin{aligned} \mathbf{repara}(r, \mathbf{node}(n, m, f, p)) &= \mathbf{node}(n, m, f, r(p)) && \text{(repara node)} \\ \mathbf{repara}(r, \mathbf{edges}(n)) &= \mathbf{edges}(n) && \text{(repara edges)} \\ \mathbf{repara}(r, \mathbf{swap}(n, m)) &= \mathbf{swap}(n, m) && \text{(repara swap)} \\ \mathbf{repara}(r, \mathbf{before}(n, k, m, (p, q))(x, y)) &= \mathbf{before}(n, k, m, r((p, q)))(x, y) && \text{(repara before)} \\ \mathbf{repara}(r, \mathbf{beside}(n, m, h, k, (p, q))(x, y)) &= \mathbf{beside}(n, m, h, k, r((p, q)))(x, y) && \text{(repara beside)} \\ \mathbf{repara}(r, \epsilon) &= \epsilon && \text{(repara empty)} \\ \mathbf{before}(\mathbf{beside}(v, w), \mathbf{beside}(x, y)) &= \mathbf{beside}(\mathbf{before}(v, x), \mathbf{before}(w, y)) && \text{(interchange law)} \\ \mathbf{beside}(\mathbf{before}(x, \epsilon), \mathbf{before}(\epsilon, y)) &= \mathbf{before}(\mathbf{beside}(x, \epsilon), \mathbf{beside}(\epsilon, y)) && \text{(dislocation law)} \\ \mathbf{before}(\mathbf{beside}(\mathbf{swap}(m, n), \mathbf{edges}(p)), \mathbf{beside}(\mathbf{edges}(n), \mathbf{swap}(m, p))) &= \mathbf{swap}(m, n + p) && \text{(swap law 1)} \\ \mathbf{before}(\mathbf{swap}(m, n), \mathbf{swap}(n, m)) &= \mathbf{edges}(m + n) && \text{(swap law 2)} \\ \mathbf{before}(\mathbf{beside}(\mathbf{edges}(m), \mathbf{swap}(n, p)), \mathbf{beside}(\mathbf{swap}(m, p), \mathbf{edges}(n))) &= \mathbf{swap}(m + n, p) && \text{(swap law 3)} \\ \mathbf{before}(\mathbf{swap}(m, n), \mathbf{before}(\mathbf{beside}(x_{(n,p)}, y_{(m,q)}), \mathbf{swap}(p, q))) &= \mathbf{beside}(y_{(m,q)}, x_{(n,p)}) && \text{(swap law 4)} \\ \mathbf{beside}(\mathbf{beside}(x, y), z) &= \mathbf{beside}(x, \mathbf{beside}(y, z)) && \text{(associativity of beside)} \\ \mathbf{before}(\mathbf{before}(x, y), z) &= \mathbf{before}(x, \mathbf{before}(y, z)) && \text{(associativity of before)} \\ \mathbf{before}(\mathbf{edges}(n), x_{(n,m)}) &= x_{(n,m)} && \text{(left unit before)} \\ \mathbf{before}(x_{(n,m)}, \mathbf{edges}(m)) &= x_{(n,m)} && \text{(right unit before)} \\ \mathbf{beside}(\epsilon, x) &= x && \text{(left unit beside)} \\ \mathbf{beside}(x, \epsilon) &= x && \text{(right unit beside)} \\ \mathbf{beside}(\mathbf{edges}(n), \mathbf{edges}(m)) &= \mathbf{edges}(n + m) && \text{(edges beside)} \end{aligned}$$

Definition A.7 (Algebraic Theory). The signature Σ from Definition A.5 together with the term equations in Definition A.6 defines an algebraic theory.

Theorem A.8. *The algebraic theory is a symmetric strict monoidal **Para**-construction.*

Proof. Note that string diagrams are directed acyclic multigraphs (DAMGs). The proof is analogous to Gibbons' proof (Gibbons, 1995) that his algebraic theory for DAMGs is an enriched symmetric strict monoidal category.

We align the constructions and laws from Definition A.2 and Definition A.4 with the term constructors of Σ in Definition A.5 and the term equations in Definition A.6. We assume a given symmetric strict monoidal $\mathcal{M}$ -actegory $(\mathcal{C}, \bullet)$, with morphisms indexed by identifiers $f, g, \dots$ and choose them together with $\mathcal{M}$ as the indices for the constructors.

- Objects A are isomorphic to identities (I, η_A) and correspond to the **edges**-constructor;
- Morphisms (P, f) correspond to the **node** (f, p) -constructor, which is indexed over the function identifier f and its parameters $p \in P$;
- The 2-morphisms $r : P' \rightarrow P$ correspond to the **repara** (r) -constructor;
- The sequential composition corresponds to the **before**-constructor;
- The horizontal and vertical composition of 2-morphisms corresponds to the repara laws;
- The monoidal product corresponds to the **beside**-constructor;
- The monoidal unit corresponds to the ϵ -constructor;
- The braiding operation corresponds to the **swap**-constructor;
- Para being a bicategory corresponds to the associativity and unit laws;
- The symmetry law corresponds to the swap laws;
- The interchange law corresponds to the interchange law.
- The dislocation law is a direct consequence of the interchange law.

□

A.3. Derived properties

The complete term rewriting system for Section 4 is the following:

Definition A.9 (Term Rewriting System). The following term rewriting system is obtained by directing the equations from Definition A.6; the dislocation law requires no rule of its own, as its directed form is an instance of the interchange rule.

$$\begin{aligned} \text{TRS} := \{ & \\ & \text{repara}(r, \text{node}(n, m, f, p)) \rightarrow \text{node}(n, m, f, r(p)); \\ & \text{repara}(r, \text{edges}(n)) \rightarrow \text{edges}(n); \\ & \text{repara}(r, \text{swap}(n, m)) \rightarrow \text{swap}(n, m); \\ & \text{repara}(r, \text{before}(n, k, m, (p, q))(x, y)) \rightarrow \text{before}(n, k, m, r((p, q)))(x, y); \\ & \text{repara}(r, \text{beside}(n, m, h, k, (p, q))(x, y)) \rightarrow \text{beside}(n, m, h, k, r((p, q)))(x, y); \\ & \text{repara}(r, \epsilon) \rightarrow \epsilon; \\ & \text{beside}(\text{before}(v, x), \text{before}(w, y)) \rightarrow \text{before}(\text{beside}(v, w), \text{beside}(x, y)); \\ & \text{before}(\text{beside}(\text{swap}(m, n), \text{edges}(p)), \text{beside}(\text{edges}(n), \text{swap}(m, p))) \rightarrow \text{swap}(m, n + p); \\ & \text{before}(\text{swap}(m, n), \text{swap}(n, m)) \rightarrow \text{edges}(m + n); \\ & \text{before}(\text{beside}(\text{edges}(m), \text{swap}(n, p)), \text{beside}(\text{swap}(m, p), \text{edges}(n))) \rightarrow \text{swap}(m + n, p); \\ & \text{before}(\text{swap}(m, n), \text{before}(\text{beside}(x_{(n,p)}, y_{(m,q)}), \text{swap}(p, q))) \rightarrow \text{beside}(y_{(m,q)}, x_{(n,p)}); \\ & \text{beside}(\text{beside}(x, y), z) \rightarrow \text{beside}(x, \text{beside}(y, z)); \\ & \text{before}(\text{before}(x, y), z) \rightarrow \text{before}(x, \text{before}(y, z)); \\ & \text{before}(\text{edges}(n), x_{(n,m)}) \rightarrow x_{(n,m)}; \\ & \text{before}(x_{(n,m)}, \text{edges}(m)) \rightarrow x_{(n,m)}; \\ & \text{beside}(\epsilon, x) \rightarrow x; \\ & \text{beside}(x, \epsilon) \rightarrow x; \\ & \text{beside}(\text{edges}(n), \text{edges}(m)) \rightarrow \text{edges}(n + m); \\ & \} \end{aligned}$$

Lemma A.10. *TRS is terminating.*

Proof. We assign to every term t over Σ a weight $\mu(t) \in \mathbb{N}$ by induction:

$$\begin{aligned} \mu(\text{node}(n, m, f, p)) &:= 2; & \mu(\epsilon) &:= 2; \\ \mu(\text{edges}(n)) &:= 3; & \mu(\text{swap}(n, m)) &:= 4; \\ \mu(\text{repara}(r, x)) &:= \mu(x) + 1; \\ \mu(\text{before}(x, y)) &:= 2\mu(x) + \mu(y); & \mu(\text{beside}(x, y)) &:= \mu(x)\mu(y) + \mu(x). \end{aligned}$$

The weight ignores the indices of the constructors. Every term has weight at least 2, and the clauses for **repara** and the compositions are strictly monotone in each term argument, so a strict decrease at the rewritten position is a strict decrease for the whole term. We check that every rule strictly decreases the weight. Each **repara** rule removes one **repara** constructor and rewrites a term of weight $\mu(t) + 1$ to a term of weight $\mu(t)$. For the four unit rules, with a the weight of the remaining argument, the left-hand side weighs $a + 6$, $2a + 3$, $2a + 2$, or $3a$, and each of these exceeds the weight a of the right-hand side. The rules without term variables rewrite fixed weights: 12 to 3 for the rules of (**swap law 2**) and (**edges beside**), and 47 and 46 to 4 for the rules of (**swap law 1**) and (**swap law 3**). For the associativity rules, with a, b, c the weights of x, y, z , the rule for **before** rewrites $4a + 2b + c$ to $2a + 2b + c$, and the rule for **beside** rewrites $abc + ac + ab + a$ to $abc + ab + a$. For the interchange rule, with a, b, c, d the weights of v, x, w, y , the left-hand side weighs $(2a + b)(2c + d) + 2a + b$, the right-hand side weighs $2ac + 2a + bd + b$, and the difference is $2ac + 2ad + 2bc$. For the rule of (**swap law 4**), with a, b the weights of $x_{(n,p)}$ and $y_{(m,q)}$, the left-hand side weighs $2ab + 2a + 12$, the right-hand side weighs $ab + b$, and the difference is $b(a - 1) + 2a + 12$. Every difference is positive, so every rewriting step decreases the weight by at least one, and a term of weight w admits at most w consecutive steps. $\square$

Consequently, every ground term rewrites to at least one normal form. Termination does not make normal forms canonical, because the term rewriting system is not confluent.

Proposition A.11 (Non-Confluence). *There is a ground term with two distinct normal forms. In particular, TRS is not confluent.*

Proof. Abbreviate $u := \text{swap}(1, 1)$ and $e := \text{edges}(2)$, and consider the term

$$t := \text{beside}(\text{before}(e, u), \text{before}(u, e)).$$

The unit rules for **before** reduce the two arguments:

$$t \rightarrow \text{beside}(u, \text{before}(u, e)) \rightarrow \text{beside}(u, u) =: t_1.$$

The interchange rule applies at the root:

$$t \rightarrow \text{before}(\text{beside}(e, u), \text{beside}(u, e)) =: t_2.$$

Both results are normal forms. Neither term contains a **repara** constructor or the constant ϵ , so the repara rules and the unit rules for **beside** do not apply. Every parallel composition in t_1 and t_2 pairs a **swap** constructor with a **swap** or **edges** constructor, so the rules for parallel compositions find neither two **edges** constructors, nor a nested parallel composition, nor two sequential compositions. The only sequential composition is the root of t_2 , and its arguments are parallel compositions, so the unit rules for **before**, the associativity rules, and the rules of (swap law 2) and (swap law 4) do not apply there. Of the two remaining swap rules, (swap law 1) requires **beside**(**swap**(m, n), **edges**(p)) as its first argument, which **beside**(e, u) is not. The rule of (swap law 3) matches the first argument **beside**(**edges**(2), **swap**(1, 1)) only with $m = 2$ and $n = p = 1$, and that match forces the second argument **beside**(**swap**(2, 1), **edges**(1)) rather than **beside**(**swap**(1, 1), **edges**(2)). Hence t_1 and t_2 are distinct normal forms of t . $\square$

The two normal forms represent the same string diagram: two crossings, side by side in one layer in t_1 , and in two layers with accompanying identity edges in t_2 . The interchange rule lets a constructor slide between the layers of a sequential composition, and different rewriting orders capture it in different layers.

Theorem A.12 (Uniqueness of encoding). *Any structure encoding of an inhabitant of the repository in Figure 9 is isomorphic to its corresponding string diagram.*

Recall the literal collections from Section 4:

- $\mathbb{L}_0$ contains literals $((f, p), n, m)$ for nodes, $((\text{swap}, n, m), n + m, n + m)$ for swaps, and $((\text{swap}, n, 0), n, n)$ for edges;
- $\mathbb{L}_1$ are sequences of literals from $\mathbb{L}_0 \cup \{\iota\}$;
- $\mathbb{L}_2$ are sequences of literals from $\mathbb{L}_1 \cup \{\iota\}$;

Proof sketch: The proof involves inductively defining a function from terms over Γ (see Section 4 for the definition) to literals of $\mathbb{L}_2$.

A function that maps the literals of $\mathbb{L}_2$ to terms over Γ can be defined by induction over $\mathbb{L}_2$.

Note that term constructors take literal arguments from $\mathbb{L}_0, \mathbb{L}_1, \mathbb{L}_2$ in a way that makes defining these functions straightforward. At last, one must prove that these functions are inverse to each other. This can be proven by structural induction, again using the literal arguments of the constructors.

B. Additional Optimization Experiments

In addition to the surrogate modeling analysis in Section 5, we evaluate the effectiveness of Bayesian optimization (BO) for exploring the synthesized search spaces. In this experiment, BO is compared against random search under the same evaluation budget. For each considered search space specification, candidate models are sampled and evaluated using the same training and testing procedure described in Section 5. Bayesian optimization uses the Gaussian process regressor

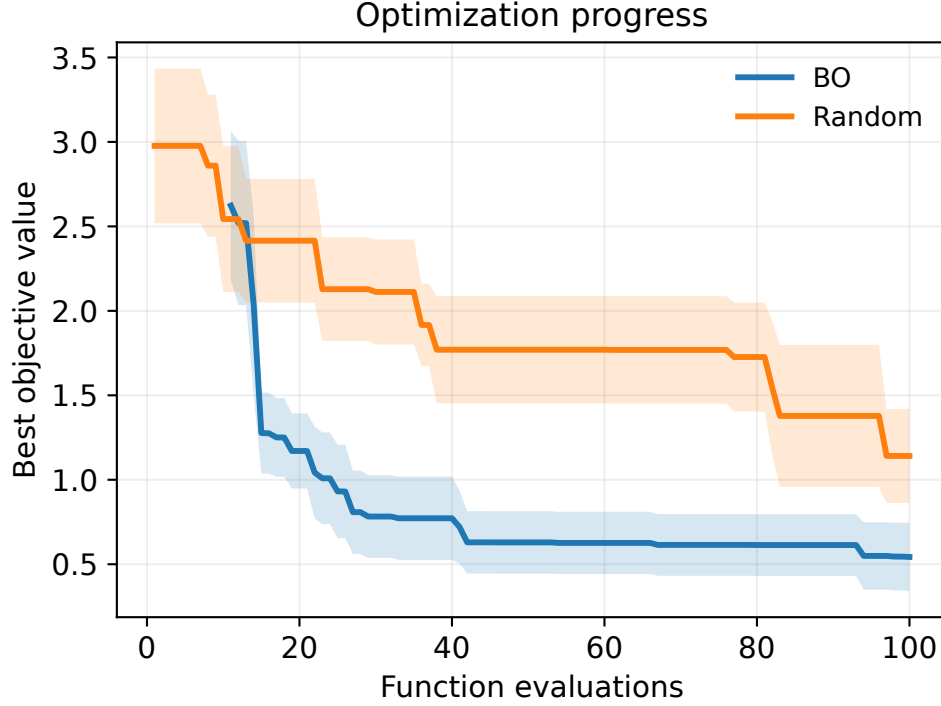


Figure 14. Optimization progress of Bayesian optimization and random search across multiple synthesized search spaces

together with an optimizable DAMG kernel defined over the synthesized model structures. Random search samples candidates uniformly from the same search space.

Figure 14 shows the optimization progress measured by the best observed objective value as a function of the number of function evaluations. Each curve aggregates optimization runs across multiple synthesized search spaces. Bayesian optimization is initialized with a sample of 10 terms; therefore its curve starts at $x = 10$, whereas random search starts at $x = 0$. Across all configurations, Bayesian optimization consistently identifies better candidates within the same evaluation budget compared to random search. This confirms that the synthesized search spaces can be effectively explored using standard surrogate-based optimization methods such as Bayesian optimization.

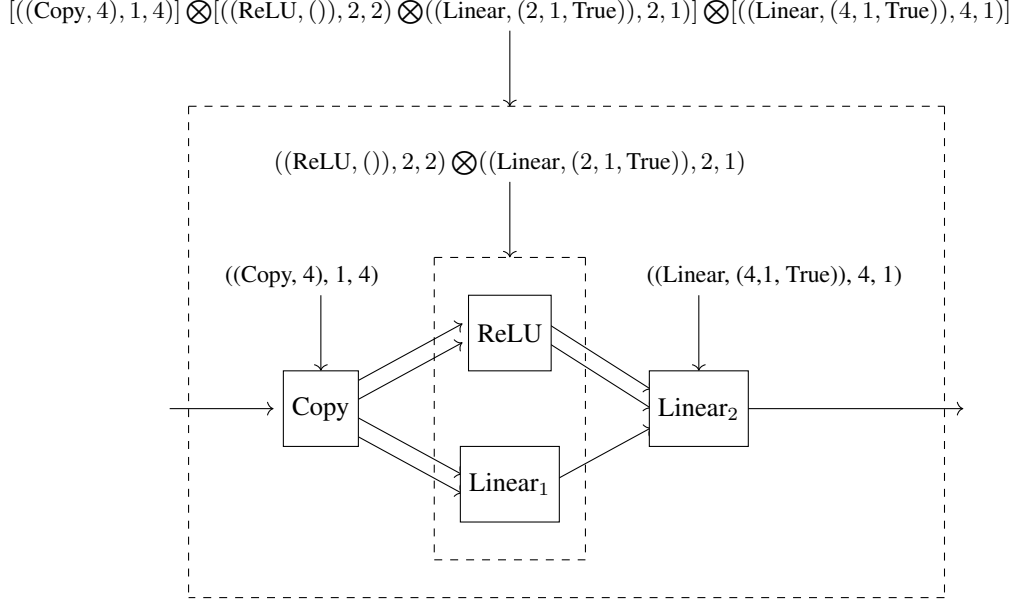


Figure 15. Best candidate discovered during experiments

C. Best Candidate Discovered by Bayesian Optimization

Figure 15 visualizes the best candidate discovered during the Bayesian optimization experiments described in Section 5. Since the structure of this candidate is known, it can be reproduced by inhabiting the following intersection type:

$$\tau := D_2(\text{IN}(1) \cap \text{OUT}(1) \cap \text{STRUCT}([((\text{Copy}, 4), 1, 4)], [((\text{ReLU}, ()), 2, 2), ((\text{Linear}, (2, 1, \text{True})), 2, 1)], [((\text{Linear}, (4, 1, \text{True})), 4, 1)]))$$

The model is represented as a string diagram corresponding to the synthesized combinatory term. The resulting architecture is surprisingly compact and consists of a small number of components. When interpreted as a PyTorch module, the model implements the function

$$f^\bullet(x) = (\max(0, x), \max(0, x), (x, x)A_1^T + b_1)A_2^T + b_2$$

where A_1 and A_2 are the weight matrices of the linear layers and b_1, b_2 are their corresponding biases. The candidate is plotted against the test data in Figure 16.

Notably, the architecture applies the ReLU activation directly to the inputs rather than to the outputs of a linear transformation. While this design may appear unusual from a conventional neural network perspective, it provides an effective approximation of the target function f^* , which contains a piecewise-linear structure with a change in slope at $x = 0$. To make this clearer, we assign the parameters of the trained model to $f^\bullet$, which leads to the function

$$f^\bullet(x) = (-1.1067 \cdot \max(0, x)) + (-0.8939 \cdot \max(0, x)) + (0.5465x + 0.4545x + 8.5420) + 1.4591$$

or equivalently written with a case distinction similar to f^*

$$f^\bullet(x) = \begin{cases} (0.5465x + 0.4545x + 8.5420) + 1.4591, & x \leq 0, \\ (-1.1067 \cdot \max(0, x)) + (-0.8939 \cdot \max(0, x)) + (0.5465x + 0.4545x + 8.5420) + 1.4591, & x > 0 \end{cases}$$

which simplifies to

$$f^\bullet(x) = \begin{cases} 1.001x + 10.0011, & x \leq 0, \\ -0.9996x + 10.0011, & x > 0. \end{cases}$$

This closely matches the target function f^* .

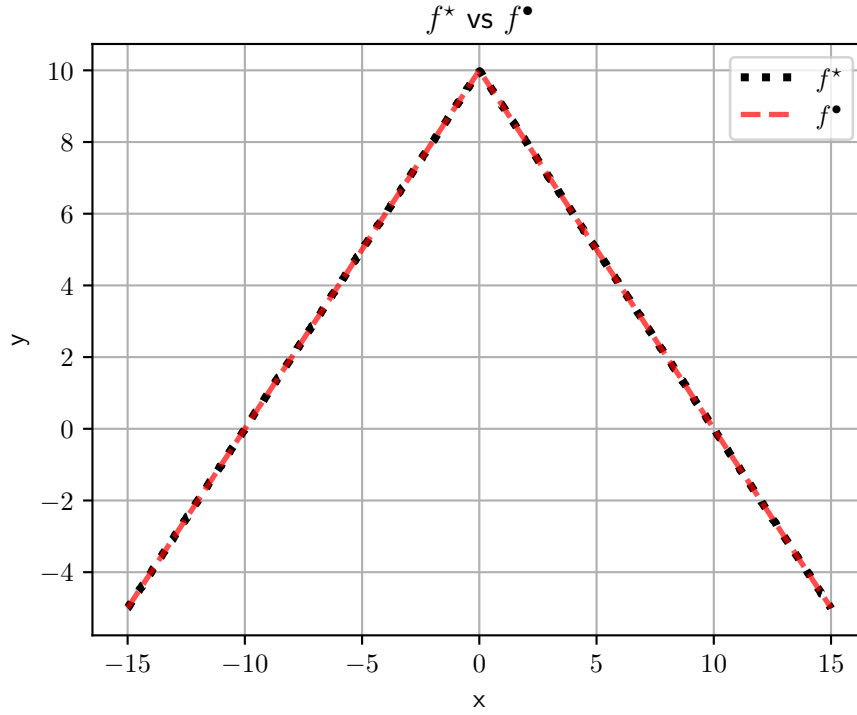


Figure 16. Original function vs. best candidate model

One may ask whether the two incoming and outgoing edges of the ReLU node are necessary. To answer this, we can easily synthesize this term by inhabiting the intersection type

$$\tau' := D_2(\text{IN}(1) \cap \text{OUT}(1) \cap \text{STRUCT}([(((\text{Copy}, 3), 1, 3)], [((\text{ReLU}, ()), 1, 1), ((\text{Linear}, (2, 1, \text{True})), 2, 1)], [((\text{Linear}, (3, 1, \text{True})), 3, 1)])))$$

and apply f_{obj} to it. The f_{obj} -value is basically the same. Therefore, during our experiments, we didn't just find a near-optimal term for f_{obj} , but also one with a near-minimal amount of parameters.

This example illustrates how the synthesized search spaces allow the discovery of compact and interpretable parametric functions that capture the underlying compositional structure of the target process.